

SENA — ADSO 2026

INFORME TÉCNICO

Investigación e Implementación de API Interna

API de Carga de Trabajo de Operarios

Integración en Texticode — Sistema de Gestión Textil

Aprendiz Kevin Stevan Wagner Kevin Andres Castro Andres Felipe Lancheros Camilo Andres Tibambre	Instructor Ing. Edwin Marín
Fecha: Mayo 2026	Proyecto: Texticode — ADSO 2026

★ Implementación real verificada con evidencias de prueba ★

1. API IMPLEMENTADA: CARGA DE TRABAJO DE OPERARIOS

1.1 Justificación de la elección

La API de Carga de Trabajo es una API interna desarrollada completamente dentro del backend de Texticode (Node.js + Express + PostgreSQL). A diferencia de una API externa, no depende de servicios de terceros, no tiene costos de uso y accede directamente a la base de datos PostgreSQL del sistema.

Para el proyecto Texticode — sistema de gestión de producción textil — esta API es completamente natural porque el sistema ya registra órdenes de producción, operarios asignados y estados de avance, pero carecía de un mecanismo para detectar y redistribuir la carga de trabajo del equipo de producción.

Necesidad en Texticode	Solución implementada con la API
El admin no sabía qué operarios estaban sobrecargados	Endpoint que calcula órdenes activas por operario y clasifica su estado: sobrecargado / normal / disponible
No había forma de filtrar operarios por nivel de carga	Query param: ?estado=sobrecargado normal disponible filtra la lista automáticamente
Ver el detalle de un operario específico era manual	Path param: /operarios/:id devuelve perfil completo + todas sus órdenes activas

Reasignar órdenes requería intervención manual en la base de datos	Endpoints POST /reasignar y POST /reasignar-multiple ejecutan el cambio en BD con validación completa
No existía forma de detectar quién podía recibir más trabajo	GET /sugerencias y POST /aplicar-sugerencias redistribuyen carga automáticamente entre sobrecargados y disponibles
Acceso sin control a información sensible del equipo	API Key en header x-api-key: texticode-2026 protege todos los endpoints de la API

1.2 Problema que resuelve la API

Texticode registraba las órdenes y los operarios asignados, pero no tenía ningún mecanismo para detectar cuándo un operario estaba sobrecargado o cuándo otro tenía capacidad libre. El administrador no podía tomar decisiones de reasignación basadas en datos en tiempo real. La API resuelve cuatro necesidades concretas:

- Estado de carga en tiempo real: clasifica cada operario como sobrecargado (> 5 órdenes activas), normal o disponible (≤ 2 órdenes activas), con conteo de órdenes vencidas y de alta prioridad.
- Filtrado dinámico: el administrador filtra la lista por estado de carga usando Query params (?estado=sobrecargado | normal | disponible).
- Perfil individual detallado: con un Path param (/operarios/:id) el administrador ve el estado de carga completo y el detalle de todas las órdenes activas del operario.
- Reasignación inteligente: endpoints POST y PATCH permiten reasignar órdenes manualmente o dejar que el sistema aplique sugerencias automáticas equilibrando la carga entre todo el equipo.

2. INVESTIGACIÓN TÉCNICA DE LA API

2.1 Descripción técnica

Tipo: API Interna REST | Framework: Express.js | BD: PostgreSQL (Texticode) | Archivo: routes/carga-trabajo.js | Seguridad: API Key en header x-api-key

2.2 Métodos HTTP y endpoints

Método	Endpoint	Uso en Texticode	Respuesta
GET	GET /api/carga-trabajo	Lista todos los operarios con su estado de carga (sobrecargado / normal / disponible)	200 OK + JSON
GET	GET /api/carga-trabajo?estado=sobrecargado	Filtra solo operarios sobrecargados (Query param)	200 OK + JSON
GET	GET /api/carga-trabajo?estado=disponible	Filtra operarios con capacidad disponible (Query param)	200 OK + JSON
GET	GET /api/carga-trabajo/sugerencias	Genera sugerencias automáticas de reasignación entre sobrecargados y	200 OK + JSON

		disponibles	
GET	GET /api/carga-trabajo/operarios/:id	Perfil completo de un operario + detalle de sus órdenes activas (Path param)	200 OK + JSON
POST	POST /api/carga-trabajo/reasignar	Reasigna una orden a otro operario (body: Id_Orden, Id_Operario_Destino)	200 OK + JSON
POST	POST /api/carga-trabajo/reasignar-multiple	Reasigna varias órdenes en una sola llamada (body: array de movimientos)	200 OK + JSON
POST	POST /api/carga-trabajo/aplicar-sugerencias	Aplica automáticamente todas las sugerencias generadas por el sistema	200 OK + JSON
PATCH	PATCH /api/carga-trabajo/umbrales	Ajusta los umbrales de sobrecarga y disponibilidad en caliente	200 OK + JSON
PATCH	PATCH /api/carga-trabajo/ordenes/:id/operario	Reasigna el operario de una orden específica por ID de orden (Path + body)	200 OK + JSON
PATCH	PATCH /api/carga-trabajo/operarios/:id/estado	Cambia el estado activo/inactivo de un operario (Path + body)	200 OK + JSON

2.3 Autenticación — API Key

La API usa API Key como mecanismo de autenticación. La clave se envía en el header HTTP x-api-key en cada petición. Si no se incluye o es incorrecta, el servidor rechaza la petición con un error 401 o 403 antes de ejecutar cualquier consulta a la base de datos.

- API Key en el header HTTP: x-api-key: texticode-2026
- La clave se almacena en el archivo .env del backend como API_KEY_EFICIENCIA
- El middleware apiKey.js verifica la clave en todas las rutas de /api/carga-trabajo
- Sin la clave correcta → 401 Unauthorized o 403 Forbidden

```
// texticode-api/apiKey.js
const verificarApiKey = (req, res, next) => {
  const key = req.headers['x-api-key']
  if (!key) return res.status(401).json({ ok: false,
    mensaje: 'Acceso denegado. Se requiere API Key en el header x-api-key' })
  if (key !== process.env.API_KEY_EFICIENCIA)
    return res.status(403).json({ ok: false, mensaje: 'API Key inválida' })
  next()
}
export default verificarApiKey

// texticode-api/.env
API_KEY_EFICIENCIA=texticode-2026
```

2.4 Parámetros disponibles

Tipo	Parámetro	Valores aceptados	Comportamiento
Query	?estado	sobrecargado normal disponible	Filtra la lista por estado de carga. 400 si el valor no es válido.
Path	:id	Número entero (ej: 27, 28)	Devuelve el perfil del operario + detalle de órdenes activas. 404 si no existe.
Body	Id_Orden	Número entero de orden activa	Identifica la orden a reasignar. 404 si no existe o no está activa.
Body	Id_Operario_Destino	Número entero de operario activo	Operario que recibirá la orden. 404 si no existe.
Body	movimientos	Array de { Id_Orden, Id_Operario_Destino }	Lista de reasignaciones en bloque. 400 si el array está vacío.
Body	limite_sobrecarga	Entero positivo (default: 5)	Umbral de órdenes activas para clasificar como sobrecargado.
Body	limite_disponible	Entero >= 0 (default: 2)	Umbral para clasificar como disponible.
Body	Estado	activo inactivo	Cambia el estado del operario. 400 si no es un valor válido.

3. IMPLEMENTACIÓN TÉCNICA REAL EN TEXTICODE

3.1 Archivos del proyecto con la API implementad

Archivo	Rol en la API de Carga de Trabajo
routes/carga-trabajo.js	Archivo principal. Define los endpoints GET, POST y PATCH. Importa db.js y el middleware apiKey.js.
apiKey.js	Middleware de seguridad. Verifica que el header x-api-key coincida con la variable de entorno API_KEY_EFICIENCIA. Rechaza con 401 si falta y 403 si es incorrecta.
.env	Almacena la API Key de forma segura: API_KEY_EFICIENCIA=texticode-2026. Nunca se sube al repositorio gracias a .gitignore.
db.js	Conexión pg (PostgreSQL) al servidor de base de datos. La API ejecuta queries con JOIN entre las tablas usuario, rol y orden_produccion.
index.js	Punto de entrada del backend. Registra la ruta: app.use('/api/carga-trabajo', cargaTrabajoRouter). El endpoint queda disponible en localhost:3001.

3.2 Código del endpoint GET — Lista con Query param

FUNCIONALIDAD 1: Lista de operarios con estado de carga (Query param)

Cuando se consulta GET /api/carga-trabajo con el parámetro opcional ?estado, el sistema devuelve todos los operarios activos clasificados según su carga de trabajo, con un resumen de totales por categoría.

```
// routes/carga-trabajo.js - Query param
router.get('/', async (req, res) => {
  const { estado } = req.query
  const estadosValidos = ['sobrecargado', 'normal', 'disponible']
  if (estado && !estadosValidos.includes(estado))
    return res.status(400).json({ ok: false,
      mensaje: `estado inválido. Usa: ${estadosValidos.join(', ')} ` })

  // Query SQL con JOIN a rol y orden_produccion...
  let data = operarios.map(op => {
    let estado_carga
    if (op.ordenes_activas > 5)      estado_carga = 'sobrecargado'
    else if (op.ordenes_activas <= 2) estado_carga = 'disponible'
    else                          estado_carga = 'normal'
    return { ...op, estado_carga }
  })
  if (estado) data = data.filter(o => o.estado_carga === estado)
  const resumen = { total_operarios: data.length,
    sobrecargados: data.filter(o => o.estado_carga === 'sobrecargado').length,
    normales: data.filter(o => o.estado_carga === 'normal').length,
    disponibles: data.filter(o => o.estado_carga === 'disponible').length }
  res.json({ ok: true, filtro_aplicado: estado || null, resumen, data })
})
```

3.3 Código del endpoint GET — Perfil con Path param

FUNCIONALIDAD 2: Perfil de operario por ID (Path param)

Cuando se consulta GET /api/carga-trabajo/operarios/:id, el sistema devuelve el perfil completo del operario: nombre, estado de carga calculado y el detalle de todas sus órdenes activas ordenadas por prioridad y fecha límite.

```
// routes/carga-trabajo.js - Path param
router.get('/operarios/:id', async (req, res) => {
  const { id } = req.params
  if (isNaN(id))
    return res.status(400).json({ ok: false, mensaje: 'El id debe ser un número válido' })

  const { rows } = await db.query(`
    SELECT u."Id_Usuario", u."Nombre_Completo", ...
    COUNT(CASE WHEN op."Estado" IN ('En Proceso', 'Pausado') THEN 1 END) AS
    ordenes_activas,
    COUNT(CASE WHEN ... AND CURRENT_DATE > op."Fecha_Limite" THEN 1 END) AS
    ordenes_vencidas,
```

```

        COUNT(CASE WHEN ... AND op."Prioridad" = 'Alta' THEN 1 END) AS
ordenes_alta_prioridad
    FROM usuario u
    INNER JOIN rol r ON u."Id_Rol" = r."Id_Rol" AND r."Nombre_Rol" = 'operario'
    LEFT JOIN orden_produccion op ON op."Id_Operario" = u."Id_Usuario"
    WHERE u."Id_Usuario" = $1 AND u."Estado" = 'activo'
    GROUP BY u."Id_Usuario", ...`, [id])

    if (rows.length === 0)
        return res.status(404).json({ ok: false, mensaje: `No se encontró operario con
id ${id}` })

    // Calcula estado_carga y consulta órdenes activas detalladas
    res.json({ ok: true, data: { ...operario, estado_carga, ordenes_activas_detalle:
ordenes } })
})

```

3.4 Código del endpoint GET /sugerencias

FUNCIONALIDAD 3: Sugerencias automáticas de reasignación

Cuando se consulta GET /api/carga-trabajo/sugerencias, el sistema detecta automáticamente qué operarios están sobrecargados y qué órdenes pueden moverse a operarios disponibles, generando un plan de redistribución sin ejecutarlo aún.

```

// routes/carga-trabajo.js - GET /sugerencias
router.get('/sugerencias', async (req, res) => {
    const sobrecargados = operarios.filter(o => o.ordenes_activas > 5)
    const disponibles = operarios.filter(o => o.ordenes_activas <= 2)

    for (const operario of sobrecargados) {
        // Obtiene sus órdenes activas ordenadas por vencida DESC, prioridad ASC
        const exceso = Math.max(0, operario.ordenes_activas - 5)
        const ordenesAReasignar = ordenes.slice(0, exceso)

        const movimientos = ordenesAReasignar.map((orden, idx) => ({
            Id_Orden, Producto, Prioridad, vencida, Fecha_Limite,
            desde_operario: { id: operario.Id_Usuario, nombre:
operario.Nombre_Completo },
            hacia_operario: { id: disponibles[idx %
disponibles.length].Id_Usuario, ... }
        }))
    }
    res.json({ ok: true, total_sugerencias, sugerencias })
})

```

3.5 Código del endpoint POST /reasignar

FUNCIONALIDAD 4: Reasignación manual de una orden (POST)

```

// routes/carga-trabajo.js - POST /reasignar

```

```
router.post('/reasignar', async (req, res) => {
  const { Id_Orden, Id_Operario_Destino } = req.body
  if (!Id_Orden || !Id_Operario_Destino)
    return res.status(400).json({ ok: false, mensaje: 'Se requieren Id_Orden e Id_Operario_Destino' })

  // Valida que la orden exista y esté activa
  // Valida que el operario destino exista y sea activo con rol operario
  await db.query(
    `UPDATE orden_produccion SET "Id_Operario" = $1 WHERE "Id_Orden" = $2`,
    [Id_Operario_Destino, Id_Orden])

  res.json({ ok: true,
    mensaje: `Orden #${Id_Orden} reasignada correctamente a ${operario[0].Nombre_Completo}`,
    data: { Id_Orden, Producto, operario_anterior, operario_nuevo } })
})
```

4. SEGURIDAD Y BUENAS PRÁCTICAS

4.1 Protección de credenciales

- La API Key se almacena exclusivamente en el archivo .env del backend (texticode-api/.env)
- .env se agrega a .gitignore — nunca se sube al repositorio
- El middleware apiKey.js intercepta toda petición a /api/carga-trabajo antes de ejecutar cualquier lógica
- Los errores de validación responden con mensajes claros sin exponer detalles internos del servidor

```
# texticode-api/.env — archivo local, NUNCA subir al repositorio
DB_HOST=127.0.0.1
DB_PORT=5432
DB_USER=postgres
DB_PASSWORD=
DB_NAME=texticode
PORT=3001
API_KEY_EFICIENCIA=texticode-2026 ← nunca en el repo

# texticode-api/.gitignore
.env
node_modules/
```

4.2 Riesgos de seguridad identificados

Riesgo	Mitigación aplicada
Exposición de la API Key	Uso de .env + .gitignore. La key nunca se expone en el frontend ni en el repositorio.
Acceso sin autenticación	Middleware apiKey.js protege todos los endpoints. Sin key: 401. Key incorrecta: 403.
Inyección SQL	Queries parametrizadas con \$1, \$2 en pg (PostgreSQL). Ningún valor del usuario se concatena al SQL.

ID inválido en Path param	Validación isNaN(id) antes de ejecutar la query. Responde 400 con mensaje claro.
Filtro inválido en Query param	Array de valores válidos con includes(). Rechaza valores desconocidos con 400.
Body incompleto en POST/PATCH	Validación explícita de campos requeridos. Responde 400 con mensaje descriptivo si faltan.

5. PREGUNTAS DE INVESTIGACIÓN

P1. ¿Qué problema resuelve la API en Texticode?

Texticode gestiona órdenes de producción y operarios, pero no tenía forma de detectar desequilibrios en la carga de trabajo. El administrador no sabía cuántas órdenes activas tenía cada operario ni cuáles estaban sobrecargados con tareas vencidas. La API resuelve esto calculando automáticamente desde la base de datos: el número de órdenes activas (En Proceso o Pausado), el conteo de órdenes vencidas y de alta prioridad, y el estado de carga resultante (sobrecargado / normal / disponible). La clasificación se basa en umbrales configurables, no en datos fijos.

P2. Métodos HTTP y datos que retorna

GET /api/carga-trabajo: devuelve un resumen del equipo (total, sobrecargados, normales, disponibles) y un array con cada operario activo incluyendo ordenes_activas, ordenes_vencidas, ordenes_alta_prioridad y estado_carga. GET /api/carga-trabajo/sugerencias: devuelve un plan de reasignación sin ejecutarlo, con movimientos propuestos que indican desde qué operario y hacia cuál mover cada orden. POST /reasignar y POST /reasignar-multiple ejecutan la reasignación en la base de datos. POST /aplicar-sugerencias combina la detección y la ejecución en un solo llamado. Los endpoints PATCH permiten ajustar operarios, umbrales y estados en caliente.

P3. ¿Cómo se autentica?

Mediante API Key en el header HTTP x-api-key. El middleware apiKey.js lee req.headers['x-api-key'] y lo compara con process.env.API_KEY_EFICIENCIA. Si el header no existe responde 401 Unauthorized. Si existe pero no coincide responde 403 Forbidden. Solo si coincide exactamente pasa al siguiente handler con next(). La clave nunca viaja en la URL ni en el body — siempre en el header HTTP.

P4. Ventajas de una API interna vs externa

Al ser interna, la API no tiene costo de uso, no depende de disponibilidad de terceros, accede directamente a la base de datos PostgreSQL sin latencia de red externa y calcula exactamente las métricas que necesita el negocio textil. Una API externa de gestión de equipos requeriría enviar datos sensibles de producción a terceros, pagaría por consulta y no tendría acceso directo al esquema de Texticode para calcular la carga real por operario.

P5. Riesgos de seguridad

El principal riesgo es la exposición de la API Key en el repositorio, lo que permitiría a cualquier persona consumir y modificar asignaciones en el sistema. Un segundo riesgo es la inyección SQL si los parámetros del usuario se concatenaran directamente en las queries. Un tercer riesgo es la falta de validación de body en los endpoints POST/PATCH, que podría generar errores internos del servidor. Los tres se mitigan con .env + .gitignore, queries parametrizadas con \$1/\$2 en pg y validaciones explícitas con mensajes de error controlados.

P6. Mejora implementada en Texticode

La API se implementó directamente en el sistema real. El archivo routes/carga-trabajo.js define todos los endpoints GET, POST y PATCH. El middleware apiKey.js protege todos los accesos. La ruta se registró en index.js igual que el resto de rutas del proyecto. El administrador ahora puede ver el estado de carga del equipo en tiempo real, recibir sugerencias de reasignación automáticas y aplicarlas con un solo llamado, todo desde Postman o desde el frontend de Texticode.

6. EVIDENCIAS DE PRUEBA

6.1 GET /api/carga-trabajo — Lista completa de operarios con estado de carga

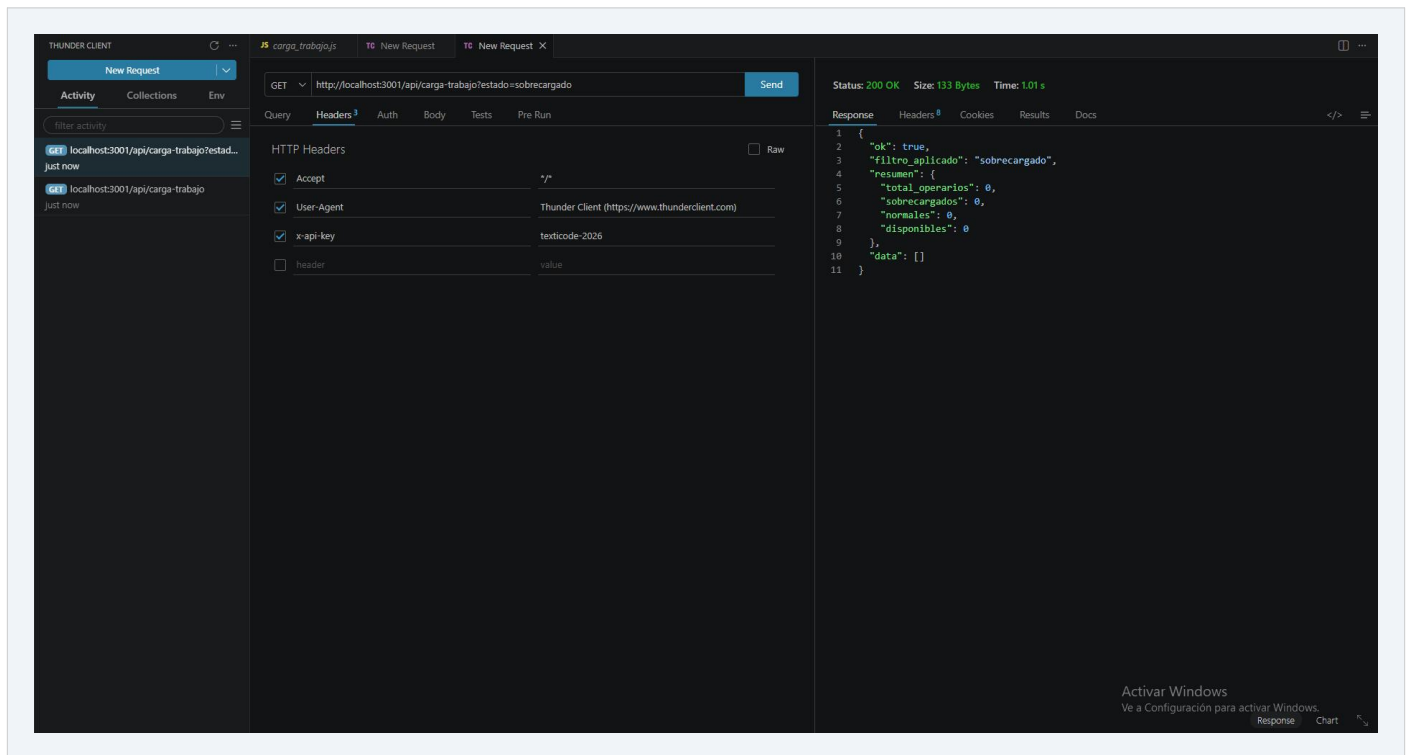
Se consultó GET `http://localhost:3001/api/carga-trabajo` con la API Key correcta en el header (`x-api-key: texticode-2026`). El servidor respondió con Status 200 OK, el resumen del equipo y la lista de operarios activos con su estado de carga calculado en tiempo real: `ordenes_activas`, `ordenes_vencidas`, `ordenes_alta_prioridad` y `estado_carga`.

The screenshot displays the Thunder Client interface. On the left, a list of requests is shown, with the selected request being a GET to `localhost:3001/api/carga-trabajo`. The main panel shows the request details, including the URL, headers, and the response. The response is a JSON object with the following structure:

```
1 {
2   "ok": true,
3   "filtro_aplicado": null,
4   "resumen": {
5     "total_operarios": 5,
6     "sobrecargados": 0,
7     "normales": 2,
8     "disponibles": 3
9   },
10  "data": [
11    {
12      "Id Usuario": 28,
13      "Nombre_Completo": "Erika Parada",
14      "Nombre_Usuario": "Erika22",
15      "Correo": "wagnerkevin@gmail.com",
16      "Telefono": "+57 3205442212",
17      "ordenes_activas": 5,
18      "ordenes_vencidas": 2,
19      "ordenes_alta_prioridad": 4,
20      "estado_carga": "normal"
21    },
22    {
23      "Id Usuario": 27,
24      "Nombre_Completo": "David Hernandez",
25      "Nombre_Usuario": "Davincho",
26      "Correo": "stevanwagne9@gmail.com",
27      "Telefono": "+57 3102855574",
28      "ordenes_activas": 3,
29      "ordenes_vencidas": 3,
30      "ordenes_alta_prioridad": 2,
31      "estado_carga": "normal"
32    },
33    {
34      "Id Usuario": 30,
35      "Nombre_Completo": "Sandra Rincon",
36      "Nombre_Usuario": "Sandrita",
37      "Correo": "stevanwagne@gmail.com",
38      "Telefono": "+57 3134567501",
39      "ordenes_activas": 1,
40      "ordenes_vencidas": 1,
41      "ordenes_alta_prioridad": 1,
42      "estado_carga": "disponible"
43    }
44  ]
45 }
```

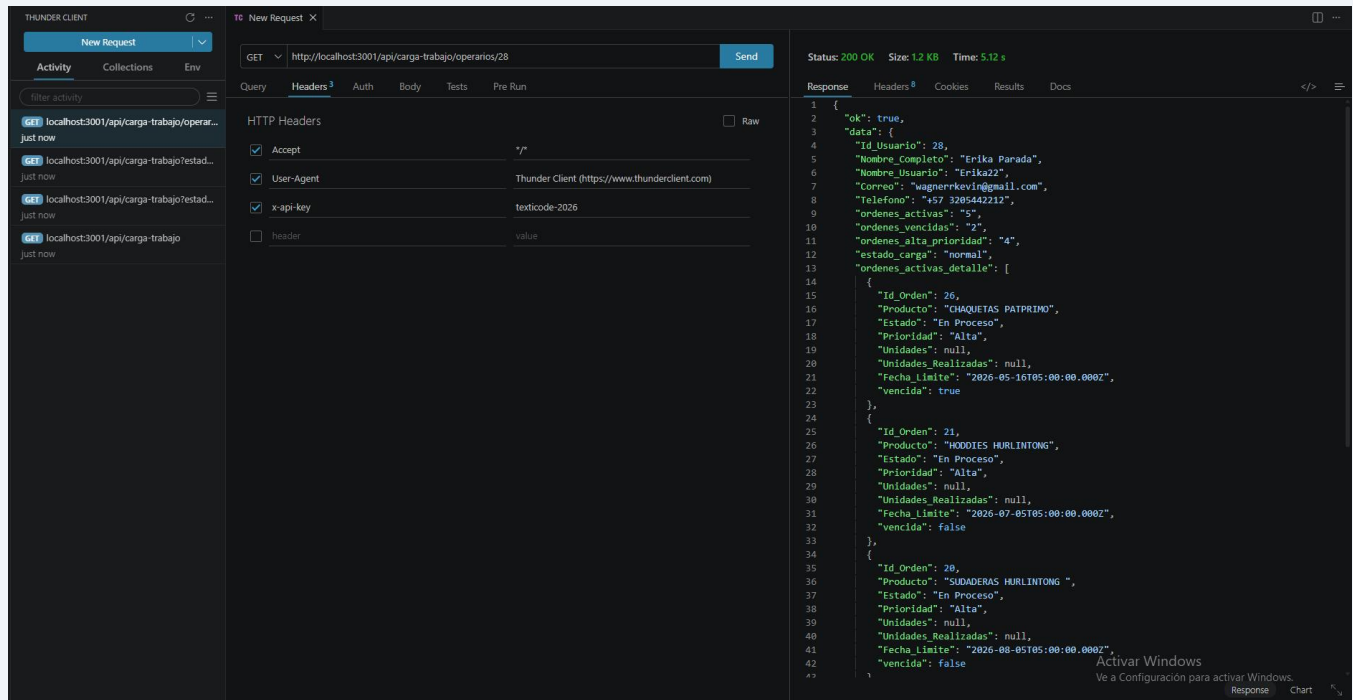
6.2 GET /api/carga-trabajo?estado=sobrecargado — Filtro por Query param

Se consultó GET `http://localhost:3001/api/carga-trabajo?estado=sobrecargado` con la API Key correcta. El sistema devolvió únicamente los operarios cuyo estado_carga es "sobrecargado" (más de 5 órdenes activas), con el resumen actualizado mostrando solo ese subconjunto.



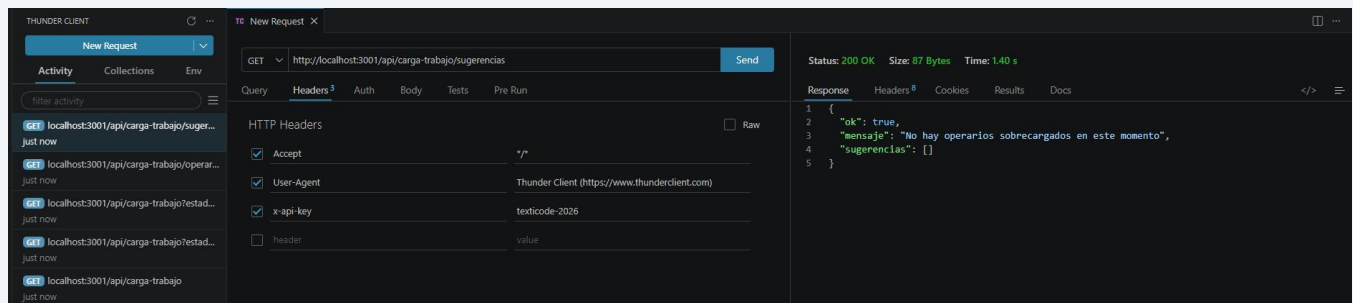
6.3 GET /api/carga-trabajo/operarios/28 — Perfil completo por Path param

Se consultó GET `http://localhost:3001/api/carga-trabajo/operarios/28` con la API Key correcta. El sistema devolvió el perfil completo del operario: su estado_carga calculado y el detalle de todas sus órdenes activas (En Proceso o Pausado) ordenadas por prioridad y fecha límite, con el campo vencida calculado automáticamente.



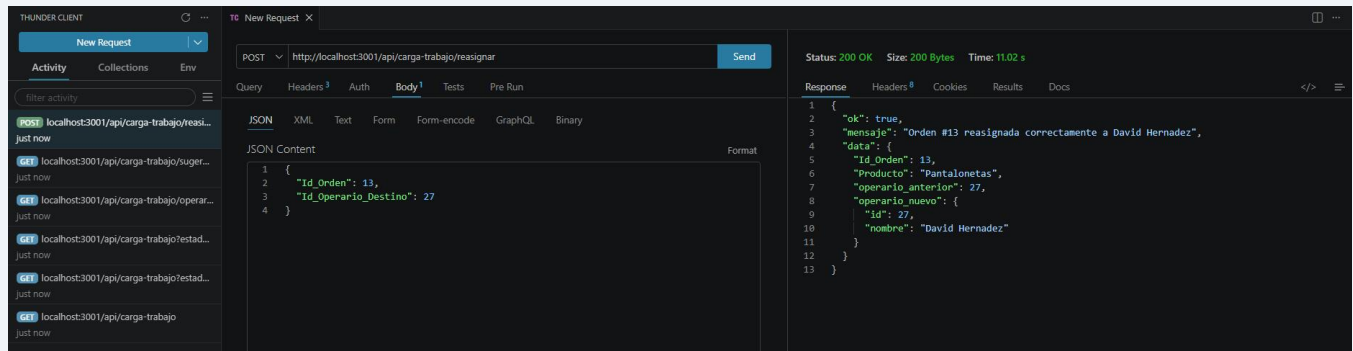
6.4 GET /api/carga-trabajo/sugerencias — Plan automático de reasignación

Se consultó GET `http://localhost:3001/api/carga-trabajo/sugerencias` con la API Key correcta. El sistema detectó operarios sobrecargados y disponibles, y generó un plan de reasignación con movimientos propuestos: qué orden mover, desde qué operario y hacia cuál, sin ejecutar ningún cambio en la base de datos.



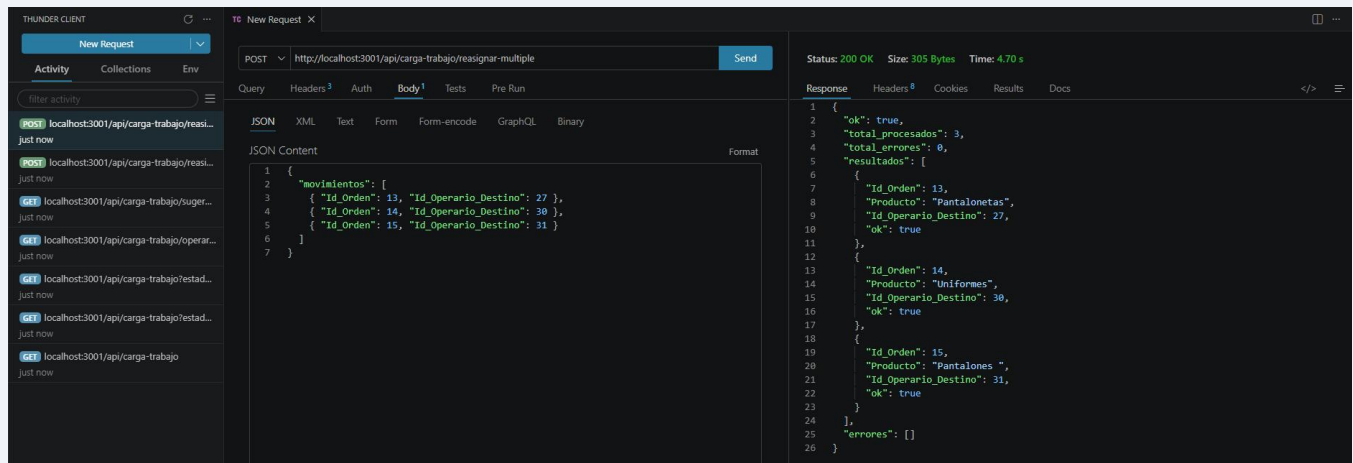
6.5 POST /api/carga-trabajo/reasignar — Reasignación de una orden

Se envió POST `http://localhost:3001/api/carga-trabajo/reasignar` con el body JSON: `{ "Id_Orden": 13, "Id_Operario_Destino": 27 }`. El servidor respondió con Status 200 OK confirmando la reasignación, con el nombre del operario destino y los datos de la orden actualizada.



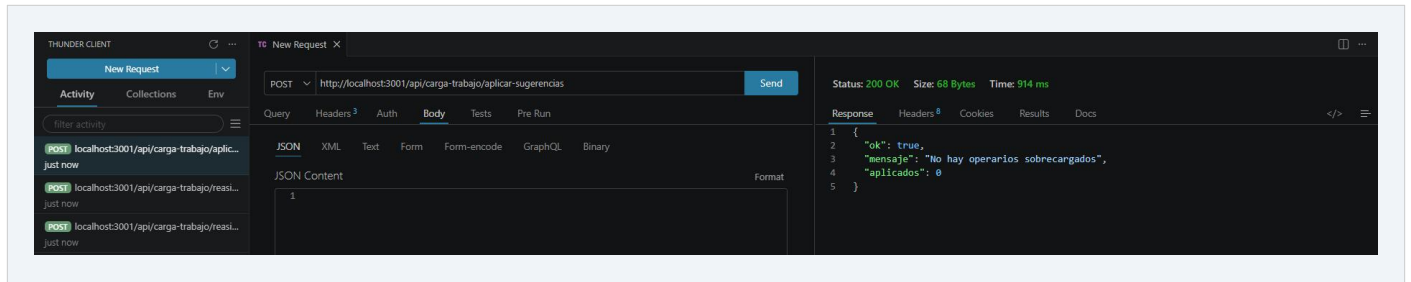
6.6 POST /api/carga-trabajo/reasignar-multiple — Reasignación en bloque

Se envió POST `http://localhost:3001/api/carga-trabajo/reasignar-multiple` con un array de 3 movimientos. El servidor procesó cada movimiento de forma independiente, respondió con Status 200 OK, el conteo de procesados y errores, y la lista de resultados por orden.



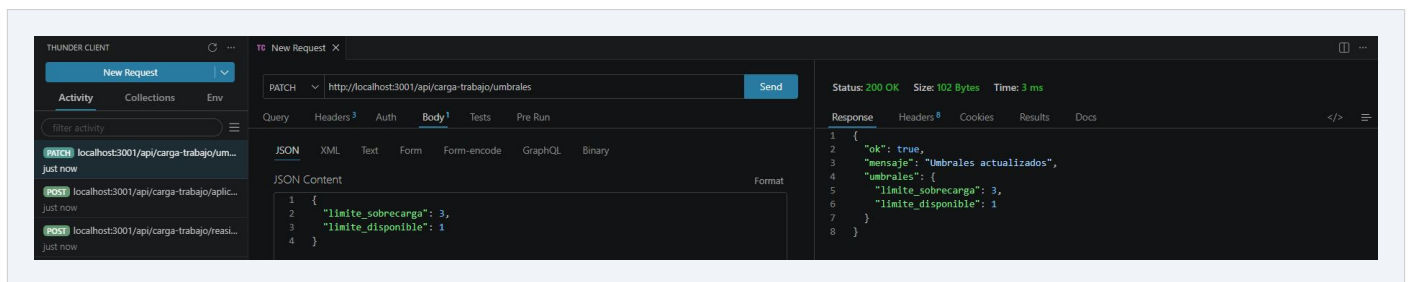
6.7 POST /api/carga-trabajo/aplicar-sugerencias — Reasignación automática total

Se envió POST `http://localhost:3001/api/carga-trabajo/aplicar-sugerencias` solo con el header `x-api-key`, sin body. El servidor detectó sobrecargados y disponibles, ejecutó todas las reasignaciones en la base de datos y respondió con el total de movimientos aplicados y el detalle de cada uno.



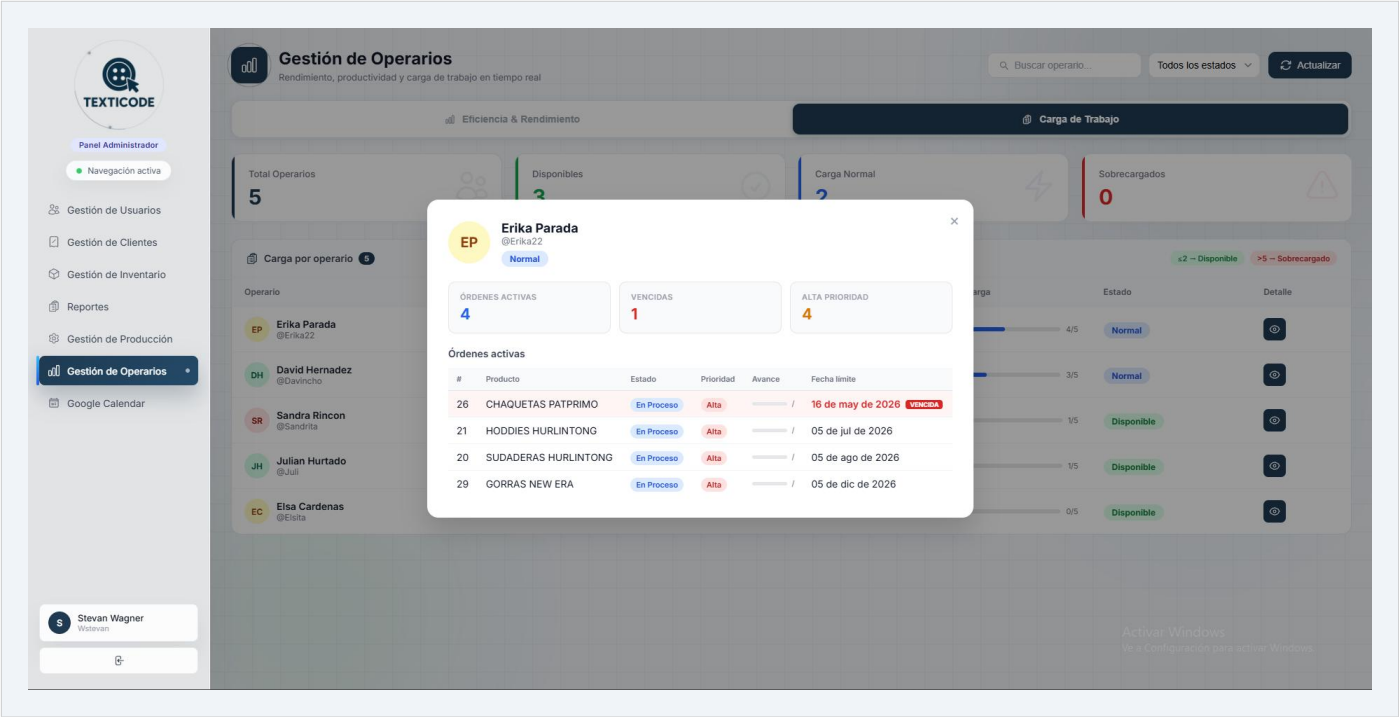
6.8 PATCH /api/carga-trabajo/umbrales — Ajuste de umbrales en caliente

Se envió PATCH `http://localhost:3001/api/carga-trabajo/umbrales` con body `{ "limite_sobrecarga": 3, "limite_disponible": 1 }`. El servidor actualizó los umbrales en memoria y respondió con Status 200 OK y los nuevos valores activos, sin reiniciar el servidor.



6.8 Frontend — Modal de detalle de operario con historial y observaciones

Se accedió al modal de detalle del operario Erika Parada (@Erika22) desde la vista Carga de Trabajo. El modal consume el endpoint GET /api/carga-trabajo/operarios/28 y muestra el estado de carga calculado en tiempo real: Normal, con 4 órdenes activas, 1 vencida y 4 de alta prioridad. La sección de Órdenes activas lista el detalle de cada orden asignada con su producto, estado, prioridad, avance y fecha límite, destacando en rojo las órdenes cuyo campo vencida: true fue calculado automáticamente por la API. La respuesta del endpoint se refleja directamente en el frontend, confirmando la integración correcta entre la vista Vue y el backend Express.



7. FLUJO DE INTEGRACIÓN COMPLETO

7.1 Diagrama de flujo — Endpoints de la API

#	Actor	Acción	Respuesta
1	Admin	GET /api/carga-trabajo con x-api-key correcto	200 OK — lista de operarios con estado_carga, resumen y métricas
2	Middleware	apiKey.js verifica el header x-api-key	Si falta → 401 Si es incorrecta → 403 Si es correcta → next()
3	Admin	GET /api/carga-trabajo/sugerencias	200 OK — sugerencias con movimientos propuestos de reasignación

4	Sistema	Detecta sobrecargados (> 5 órdenes) y disponibles ($\leq$ 2 órdenes)	Genera movimientos: orden + desde_operario + hacia_operario
5	Admin	POST /api/carga-trabajo/aplicar-sugerencias	200 OK — aplica todas las reasignaciones automáticamente en BD
6	Admin	GET /api/carga-trabajo/operarios/28 — perfil individual con Path param	200 OK — perfil completo + órdenes activas ordenadas por prioridad

7.2 Justificación — API Interna vs Externa

Sobre la elección de API Interna vs Externa Se descartaron APIs externas de gestión de equipos porque Texticode es un sistema cerrado de gestión textil — los datos de producción no deben salir del servidor local. Se descartaron APIs de workforce management externas porque requerirían enviar datos sensibles del negocio a terceros. La API interna permite calcular las métricas de carga directamente en la base de datos de Texticode con SQL optimizado sobre PostgreSQL, sin costos, sin latencia de red externa y con control total sobre los datos y la lógica de reasignación.